

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)**



Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №3
по дисциплине: «Параллельное программирование»
на тему: «Гибридное параллельное программирование с использованием
OpenMP + MPI»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей Александрович

Проверили:
доц. Островский Алексей Мичеславович,

Белгород, 2025

Цель работы: Изучить возможности гибридного подхода к параллельному программированию с использованием стандартов MPI и OpenMP, реализовать смешанную модель параллельных вычислений, оценить её эффективность по сравнению с отдельными технологиями, изучить механизмы межпроцессного взаимодействия (MPI) и многопоточного параллелизма (OpenMP).

Цель работы обуславливает постановку и решение следующих задач:

1. Ознакомиться с основами MPI + OpenMP, включая компиляцию и запуск гибридных программ.
2. Изучить особенности модели MPI и многопоточности OpenMP.
3. Научиться программировать гибридное параллельное приложение:
 - 3.1 Научиться декомпозировать вычислительную нагрузку между процессами (MPI)
 - 3.2 Научиться выполнять параллельную обработку внутри процессов (OpenMP) в условиях гибридного приложения
4. Выполнить индивидуальное задание, связанное с гибридной обработкой данных: декомпозировать задачу на межпроцессные и внутрипроцессные фрагменты, обратить внимание на балансировку нагрузки и взаимодействие между уровнями.
5. Провести экспериментальную оценку масштабируемости, потенциала ускорения и накладных расходов для различных конфигураций. Сделать выводы о применимости гибридной модели.

Условие индивидуального задания:

2. Быстрая сортировка большого массива. Разработать гибридную реализацию алгоритма быстрой сортировки. Использовать MPI_Scatter для разбиения массива между процессами. Внутри каждого процесса — использовать OpenMP (вложенные sections или tasks) для сортировки своих подмассивов. Финальную сборку отсортированных блоков выполнить в процессе-координаторе (MPI_Gather).

Ход выполнения работы

Для реализации алгоритма были определены задачи:

- Создать алгоритм быстрой сортировки (последовательный и параллельный)
- Алгоритм слияния сортированных массивов
- Применить библиотеку MPI с учётом OMP
- Покрыть расчётной информацией для анализа времени работы

Последовательная сортировка нужна с целью отсортировать часть массива которая не была определена для процесса.

Текст программы на языке C.

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <mpi.h>
#include <omp.h>

////////////////////////////////////

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

////////////////////////////////////

int partition(int *arr, int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```

void sequential_quicksort(int *arr, int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        sequential_quicksort(arr, low, pi - 1);
        sequential_quicksort(arr, pi + 1, high);
    }
}

void parallel_quicksort(int *arr, int low, int high)
{
    if (low < high)
    {
        int pivot = partition(arr, low, high);

#pragma omp task shared(arr)
        parallel_quicksort(arr, low, pivot - 1);

#pragma omp task shared(arr)
        parallel_quicksort(arr, pivot + 1, high);
    }
}

////////////////////////////////////

int *generate_full_unsorted_array(int size)
{
    int *arr = (int *)malloc(size * sizeof(int));
    for (size_t i = 0; i < size; i++)
        arr[i] = size - i;

    return arr;
}

int *generate_half_unsorted_array(int size)
{
    int *arr = (int *)malloc(size * sizeof(int));
    for (size_t i = 0; i < size; i++)
        arr[i] = i < size / 2 ? i : size - i;

    return arr;
}

int *generate_full_sorted_array(int size)
{
    int *arr = (int *)malloc(size * sizeof(int));
    for (size_t i = 0; i < size; i++)
        arr[i] = i;
}

```

```

        return arr;
    }

int is_sorted(int *arr, int size)
{
    for (int i = 0; i < size - 1; i++)
    {
        if (arr[i] > arr[i + 1])
        {
            return 0;
        }
    }
    return 1;
}

////////////////////////////////////

void merge_two_arrays(int *arr1, int n1, int *arr2, int n2)
{
    int *temp = (int *)malloc((n1 + n2) * sizeof(int));
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2)
    {
        if (arr1[i] < arr2[j])
        {
            temp[k++] = arr1[i++];
        }
        else
        {
            temp[k++] = arr2[j++];
        }
    }

    while (i < n1)
        temp[k++] = arr1[i++];
    while (j < n2)
        temp[k++] = arr2[j++];

    for (i = 0; i < n1 + n2; i++)
    {
        arr1[i] = temp[i];
    }

    free(temp);
}

void merge_all_parts(
    int *arr,
    int part_size,
    int num_parts,

```

```

    int total_size)
{
    // Обработка остатка, если общий размер не делится нацело
    int remainder = total_size % num_parts;
    if (remainder != 0)
    {
        int *remainder_data = arr + total_size - remainder;
        sequential_quicksort(
            remainder_data,
            0,
            remainder - 1);
        merge_two_arrays(
            arr,
            total_size - remainder,
            remainder_data,
            remainder);
    }

    // Сливание отсортированных частей
    for (int step = 1; step < num_parts; step *= 2)
    {
        for (int i = 0; i < num_parts; i += 2 * step)
        {
            if (i + step < num_parts)
            {
                int start = i * part_size;
                int mid = (i + step) * part_size;
                int end = (i + 2 * step) * part_size;
                if (end > total_size)
                    end = total_size;

                merge_two_arrays(
                    arr + start,
                    mid - start,
                    arr + mid,
                    end - mid);
            }
        }
    }
}

////////////////////////////////////

int main(int argc, char **argv)
{
    char mode = 'u';
    int rank, amount_proc;
    int *data = NULL;
    int *local_data = NULL;
    int n = 1000000; // Размер массива по умолчанию
    int local_n;

```

```

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &amount_proc);

if (argc > 2)
{
    n = atoi(argv[1]);
    mode = *(argv[2]);
}
local_n = n / amount_proc;

// Выделение памяти и генерация данных в процессе 0
if (rank == 0)
{
    switch (mode)
    {
        case 'u':
        case 'U':
            data = generate_full_unsorted_array(n);
            break;
        case 's':
        case 'S':
            data = generate_full_sorted_array(n);
            break;
        case 'h':
        case 'H':
            data = generate_half_unsorted_array(n);
            break;

        default:
            printf("mode \'%c\' not defined\n", mode);
            exit(0);
    }

    printf("Generated array of size %d with mode \'%c\'\n", n, mode);
}

// Распределение данных
local_data = (int *)malloc(local_n * sizeof(int));
MPI_Scatter(
    data, local_n, MPI_INT,
    local_data, local_n, MPI_INT,
    0, MPI_COMM_WORLD);

printf("Process %d received %d elements\n", rank, local_n);

// Параллельная сортировка
double start_time = MPI_Wtime();

#pragma omp parallel

```

```

    {
#pragma omp single
    {
        parallel_quicksort(local_data, 0, local_n - 1);
    }
}

double sort_time = MPI_Wtime() - start_time;
printf("Process %d sorted its part in %.4f seconds\n", rank, sort_time);

// Сбор результатов
MPI_Gather(local_data, local_n, MPI_INT,
          data, local_n, MPI_INT,
          0, MPI_COMM_WORLD);

// Финальное слияние и проверка в процессе 0
if (rank == 0)
{
    start_time = MPI_Wtime();
    merge_all_parts(data, local_n, amount_proc, n);
    double merge_time = MPI_Wtime() - start_time;

    printf("Final merge completed in %.4f seconds\n", merge_time);

    if (is_sorted(data, n))
        printf("Array is correctly sorted!\n");
    else
        printf("Error: Array is not sorted correctly!\n");

    free(data);
}

free(local_data);
MPI_Finalize();

return 0;
}

```

Протоколы, логи, скриншоты, графики.

Отчёт 1:

```

Generated array of size 300000 with mode 'u'
Process 0 received 300000 elements
Process 0 sorted its part in 122.9658 seconds
Final merge completed in 0.0000 seconds
Array is correctly sorted!

```


Отчёт 2:

```
Generated array of size 300000 with mode 'u'  
Process 1 received 150000 elements  
Process 0 received 150000 elements  
Process 0 sorted its part in 40.7463 seconds  
Process 1 sorted its part in 41.5417 seconds  
Merge completed in 0.0048 seconds  
Procedure completed in 41.5544 seconds  
Array is correctly sorted!
```

Отчёт 3:

```
Generated array of size 300000 with mode 'u'  
Process 2 received 75000 elements  
Process 3 received 75000 elements  
Process 0 received 75000 elements  
Process 1 received 75000 elements  
Process 1 sorted its part in 9.0109 seconds  
Process 0 sorted its part in 18.0187 seconds  
Process 3 sorted its part in 18.5911 seconds  
Process 2 sorted its part in 20.4632 seconds  
Merge completed in 0.0054 seconds  
Procedure completed in 20.4738 seconds  
Array is correctly sorted!
```

Отчёт 4:

```
Generated array of size 300000 with mode 'u'  
Process 0 received 75000 elements  
Process 1 received 75000 elements  
Process 2 received 75000 elements  
Process 3 received 75000 elements  
Process 2 sorted its part in 20.6436 seconds  
Process 3 sorted its part in 21.3272 seconds  
Process 1 sorted its part in 22.4496 seconds  
Process 0 sorted its part in 22.6623 seconds  
Merge completed in 0.0068 seconds  
Procedure completed in 22.7389 seconds  
Array is correctly sorted!
```

Отчёт 5:

```
Generated array of size 500000 with mode 'u'
Process 2 received 125000 elements
Process 3 received 125000 elements
Process 1 received 125000 elements
Process 0 received 125000 elements
Process 0 sorted its part in 33.7556 seconds
Process 2 sorted its part in 51.4860 seconds
Process 3 sorted its part in 57.7969 seconds
Process 1 sorted its part in 68.4646 seconds
Merge completed in 0.0097 seconds
Procedure completed in 68.5208 seconds
Array is correctly sorted!
```

```
Generated array of size 500000 with mode 'u'
Process 0 received 125000 elements
Process 1 received 125000 elements
Process 2 received 125000 elements
Process 3 received 125000 elements
Process 3 sorted its part in 40.1414 seconds
Process 0 sorted its part in 46.1682 seconds
Process 2 sorted its part in 50.0562 seconds
Process 1 sorted its part in 61.2801 seconds
Merge completed in 0.0370 seconds
Procedure completed in 61.3415 seconds
Array is correctly sorted!
```

Замеры.

Отчёты 1, 2 и 3, показывают ускорение работы алгоритма в зависимости от количества выделенных ядер. 1, 2 и 4 ядра соответственно. Логично, что чем больше ресурсов, тем меньше время выполнения.

Также в отчёте №3 замечены «провалы» по времени, из-за того, что один процесс может обработать свою часть массива быстрее остальных (из-за большего количества свободных потоков). Эту особенность можно использовать в следующей оптимизации.

Отчёт 4 показывает результат после установления опции `untied`. Для всех `omp task`, которая позволяет задачам выполняться внутри другого процесса, что выравнивает время выполнения на отдельных процессах, но время выполнения стало немного больше по сравнению с отчётом №3 (возможно из-за затрат системы на перенос задачи).

Отчёт 5 показывает в сравнении результаты реализаций, где опция `untied` только перед запуском сортировки (то есть не внутри функции сортировки), и где опция перед всеми задачами (как в отчёте 4):

```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task untied
        parallel_quicksort(local_data, 0, local_n - 1);
    }
}
```

Предположительно системе не приходится множество раз обращаться к планировщику (за счёт опции `untied`). Она обращается один раз, когда запускает сортировку.

Для наглядности количество чисел в эксперименте увеличено.

Выводы

В ходе лабораторной работы было выполнено индивидуальное задание. Изучены возможности гибридного подхода к параллельному программированию с использованием стандартов MPI и OpenMP, реализована смешанную модель параллельных вычислений, оценена её эффективность по сравнению с отдельными технологиями, изучены механизмы межпроцессного взаимодействия (MPI) и многопоточного параллелизма (OpenMP).

Разделение на процессы даёт весомое преимущество в скорости работы алгоритмов. Получилось немного ускорить программу за счёт опции untied библиотеки OMP.